
Grammar Engineering

CS5012 - P2

Student ID: 200007413

1 Grammar Design

1.1 Sentence Types

There are three ways sentences can begin with this grammar. The most basic handles a noun doing something, and is handled by the pattern:

```
S -> NP VP
---
example:
S(
  NP(Ollie)
  VP(appears)
)
```

The second type of sentence is a question sentence, which enforce questions to start with phrases like *"when do"*, or *"where does"*:

```
S -> Wh-word Aux NP VP
---
example:
S(
  Wh-word(when)
  Aux(do)
  NP(Ollie and Stan)
  VP(deliver pianos)
)
```

The final sentence type are sentences beginning with a subordinate clause:

```
S -> SBar S
--
example:
S(
  SBar(When the piano slides down the stairs)
  S(Ollie appears sad)
)
```

1.2 Verb Phrases

Verb phrases contain a verb and then optionally arguments. The arguments for this grammar can be: noun phrases, allowing verbs to have both a subject and object; noun phrases with a prepositional phrase, allowing prepositions to be added to modify the way the verb affects the object; adjective phrases allowing phrases to cause change to the noun phrase, conjunctions, allowing multiple verbs to affect the subject; a noun phrase and adverb, changing the direction of the verb on the object; or a subordinate clause, adding additional context to the sentence.

```

VP -> V
VP -> V S
VP -> V NP
VP -> V PP
VP -> V JJ
VP -> V Conj VP
VP -> V NP ADV
VP -> VP SBar

```

1.3 Noun Phrases

Noun phrases are designed to contain the subject or object of a sentence. They contain: the number of elements with numerals before the nominal; a determiner and the nominal; a conjunction of two nominals; or just a single nominal or numeral.

```

NP -> Numeral Nominal
NP -> Det Nominal
NP -> Nominal
NP -> Numeral
NP -> NP Conj NP

```

1.4 Nominals

Nominals represent the actual nouns: either proper or common, or a pronoun. Additionally, they contain the adjectives related to the nominal, and allow prepositional phrases to modify the nominal.

```

Nominal -> ProperNoun | CommonNoun
Nominal -> JJs ProperNoun | JJs CommonNoun
Nominal -> Pron
Nominal -> Nominal PP

```

1.5 Prepositional Phrases

Prepositional phrases add additional information to verbs or nominals. They begin with a preposition, and then noun phrase.

```

PP -> Prep NP

```

examples:

```

S(
  NP(Ollie and Stan)
  VP(
    V(deliver)
    NP(
      Nominal(
        Nominal(black pianos)
        PP(
          Prep(in)
          NP(the morning)
        )
      )
    )
  )
)
S(
  NP(the piano)

```

```

VP(
  V(remains)
  PP(
    Prep(on)
    NP(the steps)
  )
)
)
)

```

1.6 Adjectives

Adjectives are encoded to allow for one or many to exist before nouns or after verbs. This is done through a recursive grammar feature:

```

JJs -> JJ JJs | JJ
JJ -> "big" | "heavy" | ...

```

1.7 Agreement

Agreement is achieved through the addition of a NUM attribute to sentences and sub-sentences. Where a singular noun exists, the NUM attribute is sg, or pl for plural nouns (e.g. *"tables"*, or *"Ollie and Stan"*).

```

NP[NUM=pl] -> NP Conj NP

Pron[NUM=sg] -> 'his'
ProperNoun[NUM=sg] -> 'Stan' | 'Ollie'
CommonNoun[NUM=pl] -> 'pianos' | 'steps'
CommonNoun[NUM=sg] ->
  'piano' | 'step' | 'hat' | 'policeman' | 'morning' | 'hook'

```

The first NP rule above demonstrates that sentences must have plural verbs when two noun phrases are joined with a conjunction. Not enforcing the type of noun for these phrases allows for sentences like *"Stan and the policemen..."* to be accepted, as well as *"Stan and Ollie"*.

When singular nouns are used and no conjunction exists, verbs should be in their plural form. For example: *"Stan nods"* is valid, but *"Stan nod"* is invalid.

However, some verbs have plural forms which are the same as their third person singular version: *"Stan put his hat on"* and *"Stan puts his hat on"* are both valid.

This is encoded by separating definitions of different verb types with relevant NUM values:

```

V[NUM=pl] -> "put"
V[NUM=sg] -> "puts" | "put"
V[NUM=pl] -> "lift" | "nod"
V[NUM=sg] -> "lifts" | "nods" | "hangs"

V[NUM=sg] ->
  "appears" | "remains" | "hangs" | "slides" | "claims" | "does" | "is"
V[NUM=sg] -> "appears" | "remains" | "hangs" | "claims" | "does" | "is"
V[NUM=sg] -> "is" | "remains" | "appears"

V[NUM=pl] -> 'deliver' | 'do'
V[NUM=pl] -> "are"

```

We ensure that the NP and VP are in agreement through the addition of agreement rules:

```
S[NUM=?n] -> NP[NUM=?n] VP[NUM=?n, SUBCAT=nil]
```

Additional agreement is required for noun phrases where numerals are added to nominals to prevent phrases like *"one pianos"*, but accept *"one piano"*.

```
NP[NUM=?n] -> Numeral[NUM=?n] Nominal[NUM=?n]
Numeral[NUM=sg] -> "one"
```

Questions are a specific case, where *"where does Ollie put the piano"* is valid, but *"where does Ollie puts the piano"* and *"where do Ollie put the piano"* are invalid, because the auxiliary phrase must agree with the noun phrase, but the verb must be in its infinitive form.

```
S[NUM=?n] -> Wh-word Aux[NUM=?m] NP[NUM=?m] VP[NUM=?n, SUBCAT=nil]
Aux[NUM=sg] -> "does"
Aux[NUM=pl] -> "do"
```

1.8 Verb Sub-Categorization

Verb subcategorization ensures that certain verbs to be followed by noun phrases or prepositional phrases. For example: *"Stan puts"* is invalid because *"puts"* requires an object and direction, so *"Stan puts the piano down"* is valid.

This is handled through verb categories:

```
V[SUBCAT=nil] ->
  "appears" | "remains" | "hangs" | "slides" | "claims" | "does" | "is"
V[SUBCAT=[HEAD=s, TAIL=nil]] ->
  "appears" | "remains" | "hangs" | "slides" | "claims" | "does" | "is"
V[SUBCAT=[HEAD=pp, TAIL=nil]] ->
  "appears" | "remains" | "hangs" | "claims" | "does" | "is"
V[SUBCAT=[HEAD=pp, TAIL=[HEAD=jj, TAIL=nil]]] ->
  "appears" | "remains" | "hangs" | "claims" | "does" | "is"
V[SUBCAT=[HEAD=jj, TAIL=nil]] -> "is" | "remains" | "appears"

V[SUBCAT=[HEAD=pp, TAIL=nil]] -> "put"
V[SUBCAT=[HEAD=np, TAIL=[HEAD=adv, TAIL=nil]]] -> "put"
V[SUBCAT=[HEAD=np, TAIL=[HEAD=adv, TAIL=nil]]] -> "puts" | "put"

V[SUBCAT=nil] -> "lifts" | "nods"
V[SUBCAT=[HEAD=np, TAIL=nil]] -> "lifts" | "nods" | "hangs"
V[SUBCAT=[HEAD=np, TAIL=nil]] -> "lift" | "nod"

V[SUBCAT=nil] -> 'deliver' | 'do'
V[SUBCAT=[HEAD=np, TAIL=nil]] -> 'deliver' | 'do'

V[SUBCAT=[HEAD=np, TAIL=nil]] -> "are"
V[SUBCAT=[HEAD=jj, TAIL=nil]] -> "are"
```

Some verbs are able to not be followed by anything, such as *appears, remains, hangs, etc.* Some can be followed by adjectives such as *"Ollie appears sad"*, some can be followed by a subordinate clause such as *"Stan claims pianos are heavy"*, where *"pianos are heavy"* is the subordinate clause.

When a verb phrase is parsed, the category of verb is encoded in, this ensures that only valid matching verb sub categories match the sentence construction:

```

VP[SUBCAT=?args] -> V[SUBCAT=?args]
VP[SUBCAT=?rest] -> V[SUBCAT=[HEAD=s, TAIL=?rest]] S
VP[SUBCAT=?rest] -> V[SUBCAT=[HEAD=np, TAIL=?rest]] NP
VP[SUBCAT=?rest] -> V[SUBCAT=?rest] PP
VP[SUBCAT=?rest] -> V[SUBCAT=[HEAD=jj, TAIL=?rest]] JJ
VP[SUBCAT=?rest] -> V[SUBCAT=?rest] Conj VP[SUBCAT=?rest]
VP[SUBCAT=?rest] ->
    V[SUBCAT=[HEAD=np, TAIL=[HEAD=adv, TAIL=?rest]]] NP ADV

```

2 Critical Analysis

All positive examples were accepted and negative examples rejected by this grammar. The grammar successfully categorizes noun phrases, prepositional phrases, and verb phrases. Additionally, agreement for the most part ensures that plural and singular nouns are followed by the correct verb form.

There are very many complex relationships and verb forms in English. Some have been encoded here, but more work could be done to sub-categorize verbs into their person, tense, and overall form more clearly.

One of the negative example: "*Stan lifts*" is potentially a valid English sentence if "*lifts*" is viewed as the activity of lifting weights. However here, it is only considered as the act of lifting, and so requires an object to be lifted.

3 Conclusion

In conclusion, this grammar successfully encodes a good number of English language rules and sentence constructions. Further work could encode more valid sentence features, and reject more invalid ones. However, one fundamental flaw with this approach to sentence validation is that semantically invalid sentences are not rejected.